



Treinamento SQL - Essentials

LUIS_DEV



Comando SQL - DDL

Comando CREATE

O comando CREATE como vimos na última aula, tem como objetivo permitir a criação de objetos dentro do banco de dados. Como vimos na última aula, o comando CREATE possibilita gerar um novo banco de dados, schemas, tabelas e etc.. A partir daqui usaremos o CREATE para gerar tabelas dentro do nosso schema de graduação no banco de dados da universidade. Mas antes vamos entender as características do CREATE TABLE.

LUIS_DEV



Comando SQL - DDL

Características:

```
create table graduacao.secao ← schema.tabela
(id_curso      varchar(8),
Atributo/coluna → sec_id      varchar(8), ← Tipo de dado do atributo
semestre      varchar(6) check (semestre in ('outono', 'inverno', 'primavera', 'verão')),
ano           numeric(4,0) check (ano > 1701 and ano < 2100), ← check e restrições de integridade
predio        varchar(15),
numero_sala   varchar(10),
id_slot_tempo varchar(4),

primary key (id_curso, sec_id, semestre, ano), ← definição do grupo de chaves primárias que geram unicidade dos registros
foreign key (id_curso) references graduacao.curso (id_curso)
on delete cascade, ← definição do grupo de chaves estrangeiras que geram as restrições de integridade de relacionamento
foreign key (predio, numero_sala) references graduacao.sala_aula (predio, numero_sala)
on delete set null
);
```



Comando SQL - DDL

Características:

Schema e tabela

Etapa na qual é definido em qual instância do banco a tabela será criada, logo deve ser escolhido um schema existente e um nome para tabela que não exista no schema selecionado, além disso, a boa prática na criação da tabela indica que utilize o underscore entre um termo e outro do nome da tabela, evitar também uso de abreviações, nomes de tabelas que pode ser utilizados como nome de coluna, como por exemplo nome_aluno.

Atributo/coluna

São as colunas que definimos na criação da tabela, nomes exclusivos, por boa prática se utiliza o termo ID nos atributos que são PKs que representa um ID gerado naturalmente ou artificialmente. Para as FKs utilizem o termo FK quando se for referenciá-las. Além disso, seguir utilizando os nomes compostos dos campos separados por underscore. Por exemplo, id_paciente, fk_id_paciente.



Comando SQL - DDL

Características:

Tipo de dado do atributo

Esta etapa na modelagem é muito importante, pois aqui devemos ter muito cuidado na definição do tipo do atributo, caso uma coluna mal modelada possa gerar problemas na consulta dos dados, tornando-a mais onerosa ou gerar um entrada de dados com a integridade violada.

Tipo de dado do SQL Server

Aqui serão os tipos de dados contidos no SQL Server com um resumo rápidos de suas características, como limites e descrição breve.



Comando SQL - DDL

Características:

Tipo de dado do SQL Server

Strings:

Tipo de Dados	Descrição	Tamanho Máximo	Tamanho (bytes)
char(n)	Tamanho fixo, completado com espaços em brancos	8,000 caracteres	Tamanho Definido
varchar(n)	Tamanho variável com limite	8,000 caracteres	2 bytes + número de caracteres
varchar(max)	Tamanho variável com limite	1,073,741,824 caracteres	2 bytes + número de caracteres
text	Tamanho variável	2GB de dados (texto)	4 bytes + número de caracteres
nchar	Tamanho fixo com espaços em brancos	4,000 caracteres	Tamanho definido x 2
nvarchar	Tamanho variável	4,000 caracteres	
nvarchar(max)	Tamanho variável	536,870,912 caracteres	
ntext	Tamanho variável	2GB de texto	
binary(n)	Tamanho fixo (binário)	8,000 bytes	
varbinary	Tamanho variável (binário)	8,000 bytes	
varbinary(max)	Tamanho variável (binário)	2GB	
image	Tamanho variável (binário)	2GB	



Comando SQL - DDL

Características:

Tipo de dado do SQL Server

Numéricos:

Tipo de Dado	Descrição	Tamanho (bytes)
bit	Número inteiro que pode ser 0, 1 ou NULL	
tinyint	Permite números inteiros de 0 a 255	1 byte
smallint	Permite números inteiros entre -32,768 e 32,767	2 bytes
int	Permite números inteiros entre -2,147,483,648 e 2,147,483,647	4 bytes
bigint	Permite números inteiros entre -9,223,372,036,854,775,808 e 9,223,372,036,854,775,807	8 bytes
	Precisão de número flutuante e número de escala. Permite número de $-10^{38} + 1$ a $10^{38} - 1$.	
decimal(p,s)	O parâmetro p indica o número total máximo de dígitos que podem ser armazenados (ambos à esquerda e à direita do ponto decimal). p deve ser um valor de 1 a 38. O padrão é 18. O parâmetro s indica o número máximo de dígitos armazenados à direita do ponto decimal. s deve ser um valor de 0 a p. O valor padrão é 0.	5-17 bytes
	Precisão de número flutuante e número de escala. Permite número de $-10^{38} + 1$ a $10^{38} - 1$.	
numeric(p,s)	O parâmetro p indica o número total máximo de dígitos que podem ser armazenados (ambos à esquerda e à direita do ponto decimal). p deve ser um valor de 1 a 38. O padrão é 18. O parâmetro s indica o número máximo de dígitos armazenados à direita do ponto decimal. s deve ser um valor de 0 a p. O valor padrão é 0	5-17 bytes
smallmoney	Tipo de "Moeda" de -214,748.3648 a 214,748.3647	4 bytes
money	Tipo de "Moeda" de -922,337,203,685,477.5808 a 922,337,203,685,477.5807	8 bytes
float(n)	Precisão de número flutuante de $-1.79E + 308$ a $1.79E + 308$. O parâmetro n indica se o campo deve conter 4 ou 8 bytes. float (24) contém um campo de 4 bytes e o float(53) mantém um campo de 8 bytes. O valor padrão de n é 53.	4 ou 8 bytes
real	Precisão de número flutuante de $-3,40E + 38$ a $3,40E + 38$	4 bytes



Comando SQL - DDL

Características:

Tipo de dado do SQL Server

Datas:

Tipo de Dado	Descrição	Tamanho (bytes)
datetime	De 1 de janeiro de 1753 a 31 de dezembro de 9999 com uma precisão de 3,33 milisegundos	8 bytes
datetime2	De 1º de janeiro de 0001 a 31 de dezembro de 9999 com precisão de 100 nanossegundos	6-8 bytes
smalldatetime	De 1 de janeiro de 1900 a 6 de junho de 2079 com precisão de 1 minuto	4 bytes
date	Armazena apenas uma data. De 1 de janeiro de 0001 a 31 de dezembro de 9999	3 bytes
time	Armazena um tempo apenas para uma precisão de 100 nanossegundos	3-5 bytes
datetimeoffset	O mesmo que datetime2 com a adição de um deslocamento de fuso horário	8-10 bytes
timestamp	Armazena um número único que é atualizado sempre que uma linha é criada ou modificada. O valor do timestamp é baseado em um relógio interno e não corresponde ao tempo real. Cada tabela pode ter apenas uma variável timestamp	

Material complementar sobre o tema:

[Doc oficial](#)

LUIS_DEV



Comando SQL - DDL

Características:

Check e restrições de integridade

A definição de integridade tem como objetivo proteger a integridade dos dados, baseada na modelagem do banco de dados, no qual usuários autorizados a alterar os dados, o fazem de forma que os dados não percam a sua consistência diante das restrições definidas.

Tipos de restrições

- not null
- unique
- Check

LUIS_DEV



Comando SQL - DDL

Características:

Definição de chaves primárias

As chaves primárias de uma tabela tem o objetivo de tornar um registro único dentro de um conjunto de registros de uma tabela. Uma pk é conhecida como uma superchave, devido a seu objetivo, a superchave pode ser formada por apenas um atributo ou grupo de atributos de uma tabela. É importante salientar que uma chave primária pode ser fruto de uma chave estrangeira de uma outra tabela. As PKs recebem restrições de unique e not null assim que as chaves primárias e a tabela são criadas.



Comando SQL - DDL

Características:

Definição de chaves estrangeiras

As chaves estrangeiras tem como papel permitir o relacionamento entre as tabelas, com isso quando uma FK é referenciada a partir de um PK é criada uma restrição relacional, no qual os dados desta coluna precisam estar dentro do grupo de dados da tabela original da chave primária. Em suma, ao preencher com dados a FK é necessário que sejam os mesmos dados da tabela original, que no caso é a PK. As chaves estrangeiras podem ser nulas de acordo com os desenhos de negócio dos banco de dados. Ademais, quando se referencia uma FK é necessário que os tipos dos atributos e precisões sejam iguais a origem, além disso é visto como uma boa prática repetir o nome da coluna e se adicionar o fk no início do nome do atributo. As fk quando referenciadas em uma tabela ela gera restrições em determinadas transações, vamos ver quais são as restrições e alternativas.



Comando SQL - DDL

Características:

Definição de chaves estrangeiras

Restrições e alternativas de ações para as fks

As chaves estrangeiras por padrão geram restrições de ações na tabela pai da fk, por padrão se uma ação violar a integridade da chave será assumido um NO ACTION, logo a transação executada será revertida. Contudo é possível especificar qual tipo de procedimento deve ser adotado em caso de deleção ou atualização dos dados sejam feitas na tabela, esses procedimentos são CASCADE, SET NULL e SET DEFAULT.



Comando SQL - DDL

Características:

Definição de chaves estrangeiras

Restrições e alternativas de ações para as fks

NO ACTION - A transação que viola a integridade é rejeitada e as alterações são revertidas.

CASCADE - As linhas correspondentes são atualizadas ou excluídas na tabela de referência quando essa linha é atualizada ou excluída na tabela pai. CASCADE não poderá ser especificada se uma coluna timestamp fizer parte da chave estrangeira ou da chave referenciada. Combinações **on delete cascade**, **on update cascade**.

SET NULL - Todos os valores que compõem a chave estrangeira são definidos como NULL quando a linha correspondente da tabela pai é atualizada ou excluída. Para que essa restrição seja executada, as colunas de chave estrangeira devem ser anuláveis. Combinações **on delete set null**, **on update set null**.



Comando SQL - DDL

Características:

Definição de chaves estrangeiras

Restrições e alternativas de ações para as fks

SET DEFAULT - Todos os valores que compõem a chave estrangeira serão definidos com seus valores padrão se a linha correspondente na tabela pai for atualizada ou excluída. Para que essa restrição seja executada, todas as colunas de chave estrangeira devem ter definições padrão. Se a coluna for anulável e não houver nenhum valor padrão explícito definido, NULL se tornará o valor padrão implícito para a coluna. Exemplo de default **column_b** **INT DEFAULT 50**.



Comando SQL - DDL

Características:

CONSTRAINT

As constraints são os grupos de restrições enumeradas e detalhadas acima, mas para além do que sabemos, qual a outra utilidade do constraint? Além de ser um conjunto de restrições, tem como funcionalidade possibilitar a nomeação da constraints que podem ser escolhidas para impor integridade aos dados, são check e unique. Por exemplo **CONSTRAINT exemplo CHECK (ColumnD > 10 AND ColumnD < 50 ou TransactionID int NOT NULL,**

CONSTRAINT AK_TransactionID UNIQUE(TransactionID)



Comando SQL - DDL

Comando DROP

O comando DROP como vimos na última aula, tem como objetivo permitir a exclusão de objetos dentro do banco de dados. E esse comando faz a exclusão de tabelas, schemas e banco de dados. É importante lembrar que o processo de exclusão também respeita as restrições nas quais a tabela é estendida, como a restrição relacional das chaves estrangeiras. Exemplos de utilização do comando **drop**

```
table graduacao.curso_pre_requisito;
```



Comando SQL - DDL

Comando TRUNCATE

O comando TRUNCATE tem como objetivo permitir a exclusão a nível de registros de uma tabela, mas diferente do comando DELETE, não é possível por uma cláusula WHERE para filtrar registros específicos. O filtro que é permitido é o de partição, só é possível aplicar esse filtro de particionamento se o banco de dados tiver essa funcionalidade ativada. Outro ponto importante é que o TRUNCATE é mais performático que o DELETE, pois faz a exclusão dos registros de uma vez, já o DELETE faz um por vez.

É importante lembrar que a transação de TRUNCATE também respeita as restrições nas quais a tabela é estendida, como a restrição relacional das chaves estrangeiras. Além disso, não será possível reverter a operação de TRUNCATE com o ROLLBACK. Exemplos de utilização do comando **truncate table graduacao.curso_pre_requisito;**



Comando SQL - DDL

Comando ALTER

O comando ALTER tem como objetivo permitir alterações a nível de tabela no nosso caso, pois o comando ALTER pode ser utilizado em vários níveis do banco de dados. Então quais os escopos pertinentes para o nosso caso? Podemos usar o ADD column, DROP column, ADD CONSTRAINT, DROP CONSTRAINT, ADD PRIMARY KEY, ADD FOREIGN KEY, DROP PRIMARY KEY, DROP FOREIGN KEY, ADD CHECK CONSTRAINT, DROP CHECK CONSTRAINT . Essas são as principais alternativas de utilização do comando ALTER.

Alguns exemplos:

-
- `ALTER TABLE graduacao_teste.instrutor ADD instituicao_formação varchar(30);`
- `ALTER TABLE graduacao_teste.instrutor ADD id_instituicao_formação numeric(4) NOT NULL;`
- `ALTER TABLE graduacao_teste.instrutor ALTER COLUMN instituicao_formação varchar(20);`
- `ALTER TABLE graduacao_teste.instrutor ALTER COLUMN instituicao_formação varchar(30) NOT NULL;`
- `ALTER TABLE graduacao_teste.instrutor DROP CONSTRAINT CK__instrutor__salar__619B8048; --DELETE DA RESTRICAO NATIVA DA CRIACAO DA PK NA CRIACAO DA TABELA`
- `ALTER TABLE graduacao_teste.instrutor ADD CONSTRAINT pk_instrutor PRIMARY KEY (id_instrutor, id_instituicao_formação);`
- `ALTER TABLE graduacao_teste.instrutor DROP CONSTRAINT pk_instrutor;`
- `ALTER TABLE graduacao_teste.instrutor DROP COLUMN id_instituicao_formação,instituicao_formação;`



Comando SQL - DML

Comando INSERT

O comando INSERT gera uma solicitação para inserir dados na tabela, essa inserção deve respeitar as restrições contidas na tabela. Logo se as colunas possuem FK, CHECK, NOT NULL, no momento da inserção dos dados as informações selecionada para preencher a tabela devem respeitar as CONSTRAINT estabelecidas. Agora que entendemos como o INSERT se comporta, vamos ver quais as possibilidades de INSERT podemos utilizar para gerar o preenchimento de dados em uma relação.

As formas para se fazer um INSERT são as seguintes:

- Declarando os atributos e valores na hora de fazer o INSERT - **insert into graduacao_teste.curso (id_curso, titulo, nome_departamento, creditos) values ('100', 'Programming', 'Comp. Sci.', 4)**
- Declarando apenas os valores que deseja inserir na tabela - **insert into graduacao.curso values ('100', 'Programming', 'Comp. Sci.', 4)**



Comando SQL - DML

Comando INSERT

As formas para se fazer um INSERT são as seguintes:

- Utilizando o resultado de uma query como origem para o INSERT

```
insert into graduacao.instrutor
```

```
select id_aluno, nome, nome_departamento, 36000 as salario
```

```
from graduacao.aluno
```

```
where nome_departamento = 'Civil Eng.';
```



Comando SQL - DML

Comando INSERT

As formas para se fazer um INSERT são as seguintes:

- Utilizando um arquivo para fazer um INSERT em massa, chamado comumente por BULK

[DOC BULK INSERT](#)



Comando SQL - DML

Comando UPDATE

O comando UPDATE gera uma solicitação para atualizar os dados de uma tabela, essa inserção deve respeitar as restrições contidas na tabela. Logo se as colunas possuem FK, CHECK, NOT NULL, no momento da atualização dos dados os valores selecionados para serem atualizados devem respeitar as CONSTRAINT estabelecidas. Agora vamos ver como podemos atualizar valores de uma tabelas.

As formas para se fazer um UPDATE são as seguintes:

- Fazendo uma atualização sem filtros no UPDATE, gerando uma mudança em todos os valores da coluna-
update graduacao.instrutor

```
set salario = salario * 1.05;
```



Comando SQL - DML

Comando UPDATE

As formas para se fazer um UPDATE são as seguintes:

- Fazendo uma atualização da cláusula WHERE no UPDATE, gerando mudanças em valores específicos de acordo com os filtros escolhidos. Além disso, pode ser utilizado subqueries para construir a condição para atualizar os dados.

```
update graduacao.instrutor
```

```
set salario = salario* 1.05
```

```
where salario < 70000;
```

```
update graduacao.instrutor
```

```
set salario = salario* 1.05
```

```
where salario < (select avg (salario)
```

```
from graduacao.instrutor)
```

```
update graduacao.aluno
```

```
set total_credits = (select sum(credits)
```

```
from graduacao.nota_aluno, graduacao.curso
```

```
where nota_aluno.id_curso = curso.id_curso and
```

```
aluno.id_aluno = nota_aluno.id_aluno
```

```
and nota_aluno.nota <> 'F'
```

```
and nota_aluno.nota is not null);
```



Comando SQL - DML

Comando UPDATE

As formas para se fazer um UPDATE são as seguintes:

- Fazendo uma atualização da cláusula CASE no UPDATE, gerando várias mudanças nos valores de acordo com as condições estabelecidas no CASE.

```
update graduacao.instrutor
```

```
set salario = case
```

```
when salario <= 100000 then salario * 1.05
```

```
else salario * 1.03
```

```
end;
```



Comando SQL - DML

Comando DELETE

O comando DELETE como vimos na última aula, tem como objetivo permitir a exclusão a nível de registros de uma tabela. E esse comando faz a exclusão dos registros de uma tabela de forma total ou mediante aos filtros de uma cláusula WHERE. É importante lembrar que a transação de exclusão de registros também respeita as restrições nas quais a tabela é estendida, como a restrição relacional das chaves estrangeiras. Além disso, se o banco de dados estiver com a configuração de COMMIT automático, será possível reverter qualquer erro usando o comando ROLLBACK. Exemplos de utilização do comando

```
delete from graduacao.professor;
```

```
delete from graduacao.professor where ano = 2007 and id_professor = '14365';
```



Comando SQL - DQL

Comando SELECT

O comando SELECT é o comando mais utilizado no ambiente de consultas é SQL, é a base para retorno dos dados de um banco de dados, a estrutura básica de uma consulta utilizando o SELECT é a **SELECT * FROM table WHERE atributo¹ >= ""**. Diante disto, as consultas têm uma infinidade de possibilidades de construção.

Consulta de uma única consulta

Diante disto, temos **consultas simples** nas quais utilizamos a estrutura básica para acessar aos dados, dentro desse tipo de consulta podemos trazer todos os campos utilizando *****, podemos selecionar apenas o campos necessários, , um adendo é visto como uma boa prática, para utilização do resultado da consulta.



Comando SQL - DQL

Comando SELECT

select nome

from graduacao.instrutor

where nome_departamento = 'Comp. Sci.';

select nome

from graduacao.instrutor

where nome_departamento = 'Comp. Sci.'

and salario > 80000;

Consulta com múltiplas tabelas

As consultas com múltiplas tabelas, funciona como um produto cartesiano que gera um retorno com todas as possibilidades de cruzamentos entre as tuplas das tabelas selecionadas, em resumo, todas os registros possíveis de serem combinados serão retornados se não for aplicado um predicado na cláusula WHERE para delimitar o retorno dentro do data frame de dados de correlação verdadeira.



Comando SQL - DQL

Comando SELECT

Consulta com múltiplas tabelas

```
select nome, id_curso  
from graduacao.instrutor , graduacao.professor  
where id_instrutor = id_professor;
```

```
select *  
from graduacao.instrutor , graduacao.professor  
where id_curso = '843';
```

```
select nome, id_curso, nome_departamento  
from graduacao.instrutor , graduacao.professor  
where id_instrutor = id_professor  
and nome_departamento = 'Pol. Sci.';
```



Comando SQL - DQL

Comando SELECT

Consulta com subconsultas

As consultas com que utilizam subconsultas, tem sua principal utilização em três formas, para servir como elemento de comparação no predicado da cláusula WHERE, outra forma de usar na cláusula FROM em substituição de uma relação e por fim na cláusula WITH no qual são geradas tabelas temporárias que vão ser utilizadas no escopo da consulta.

```
select nome_departamento, avg_salario
from (select nome_departamento,
avg (salario) as avg_salario
from graduacao.instrutor
group by nome_departamento) subquery
where avg_salario > 42000;
```

```
select *
from graduacao.secao as S
where semestre = 'outono' and ano = 2002 and
exists (select *
from graduacao.secao as T
where ano = 2007
and S.id_curso = T.id_curso);
```



Comando SQL - DQL

Comando SELECT

Consulta com subconsultas

with max_orcamento (valor) as

(select max(orcamento)

from graduacao.departamento)

select nome_departamento

from graduacao.departamento, max_orcamento

where departamento.orcamento = max_orcamento.valor;

LUIS_DEV



Material de apoio

Create Table

[Doc Create Table](#)

Drop Table

[Drop Table](#)

Alter Table

[Doc Alter table](#)

Palavras Reservadas

[Doc palavras reservadas](#)

Convenções SQL Server

[Doc convenções SQL SERVER](#)

Insert

[Doc Insert](#)

Updata

[Doc Update](#)

Delete

[Doc Delete](#)

Truncate

[Doc Truncate](#)

Select

[Doc Select](#)

Restrições

[Doc Restrições](#)

LUIS_DEV



Dúvidas?

LUIS_DEV

